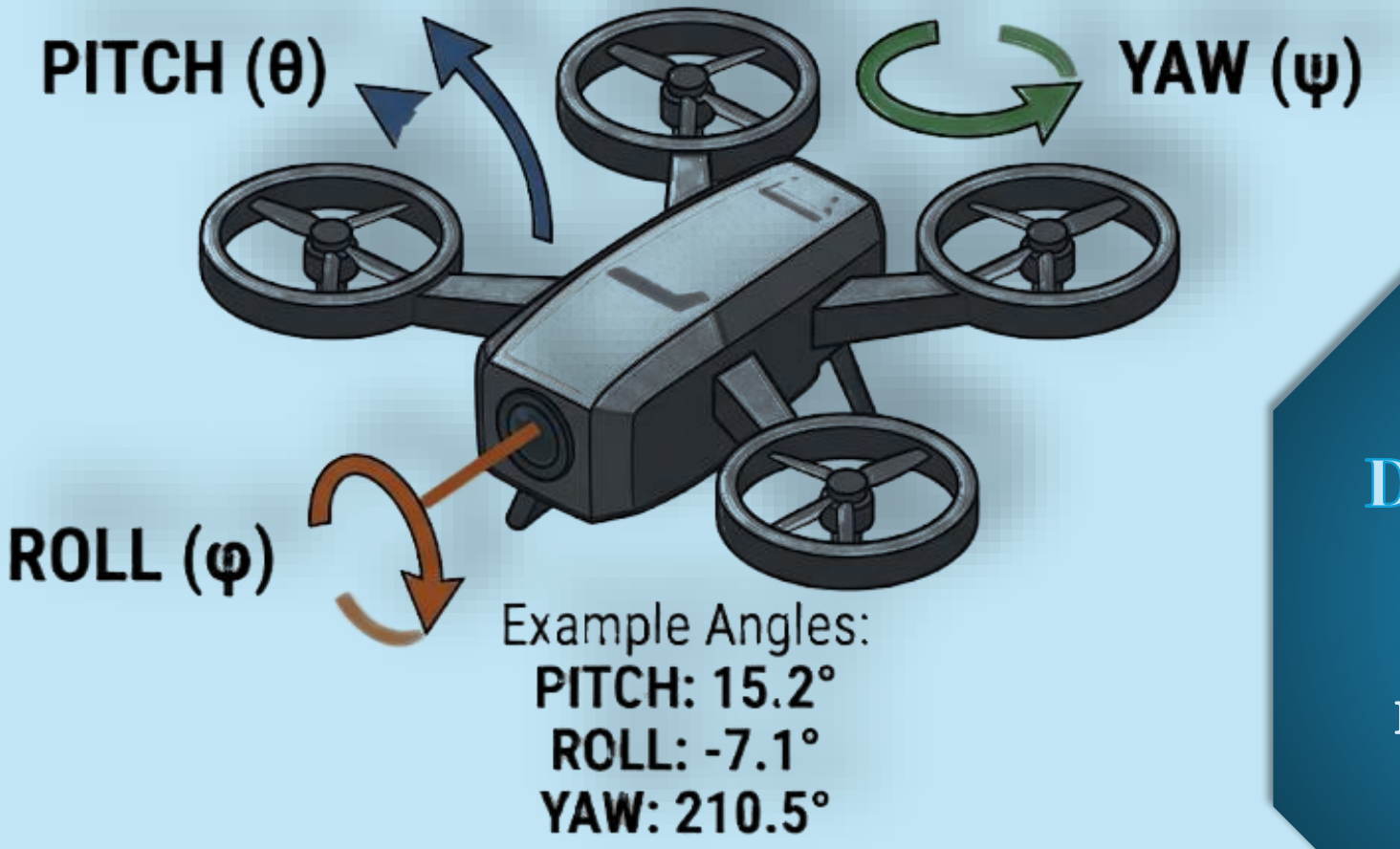
Accelerometers



Dr Bharat Verma
Control System Lab
ECE Department
The LNM Institute of Information Technology

The Design Objective

Objective

Objective

Mechanical Design and Stability

Objective

Mechanical Design and Stability

Sensors

Objective

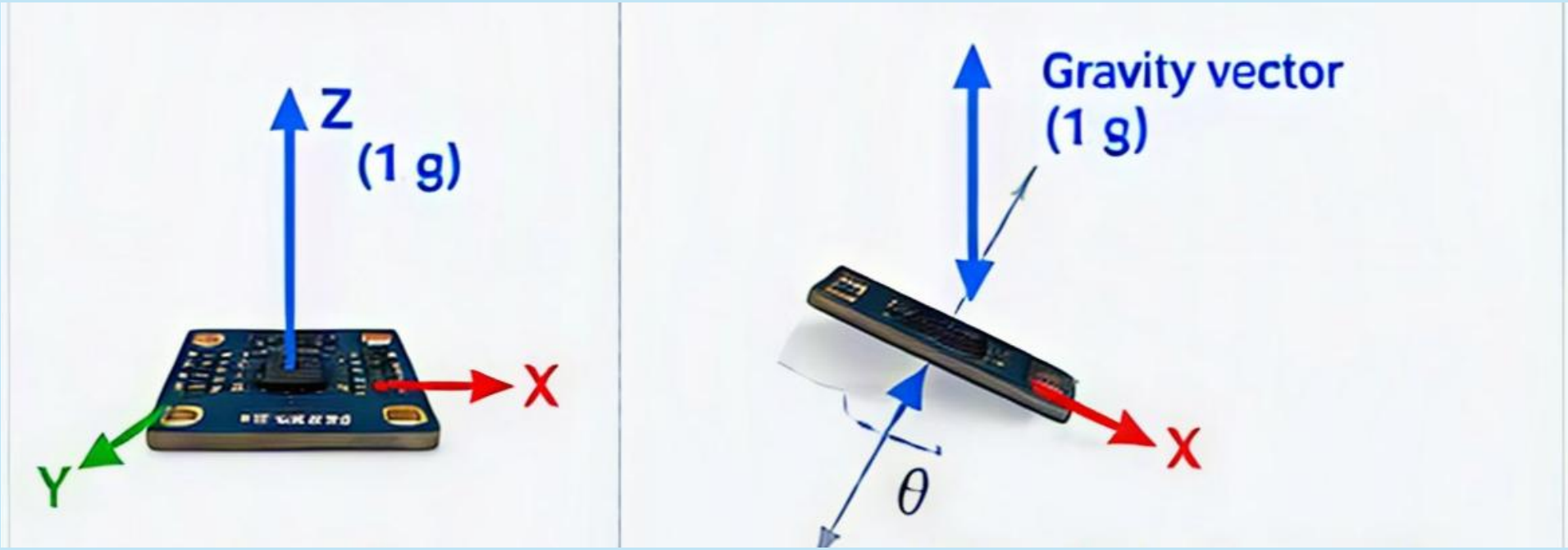
Mechanical Design and Stability

Sensors

Actuator

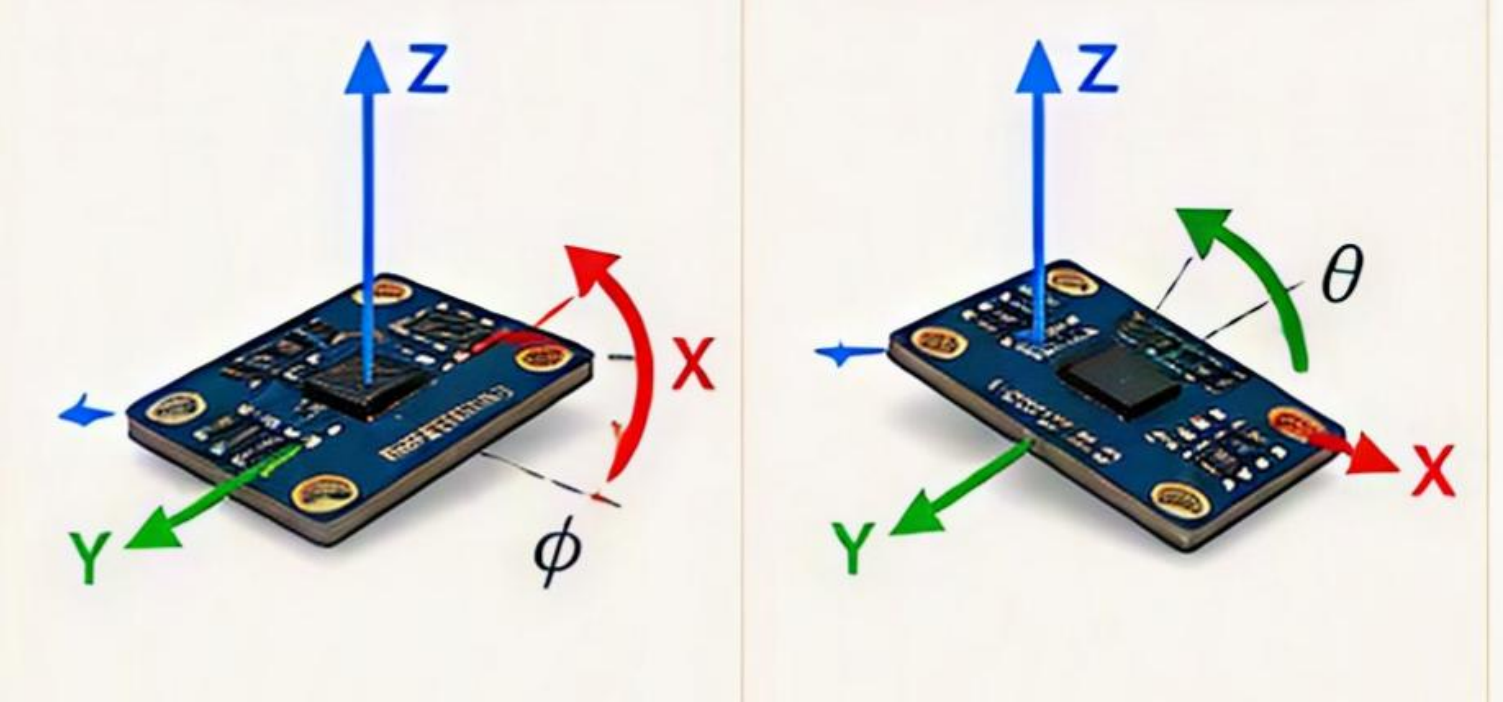
The Gravity and Measurement

3-Axis Accelerometer: *Measures* ***linear acceleration*** *(in gforces) along the X, Y, and Z axes. It detects changes in velocity, gravity*



The Gravity and Measurement

$$A_x = 0, A_y = 0 \textbf{ and } A_z = 1g$$



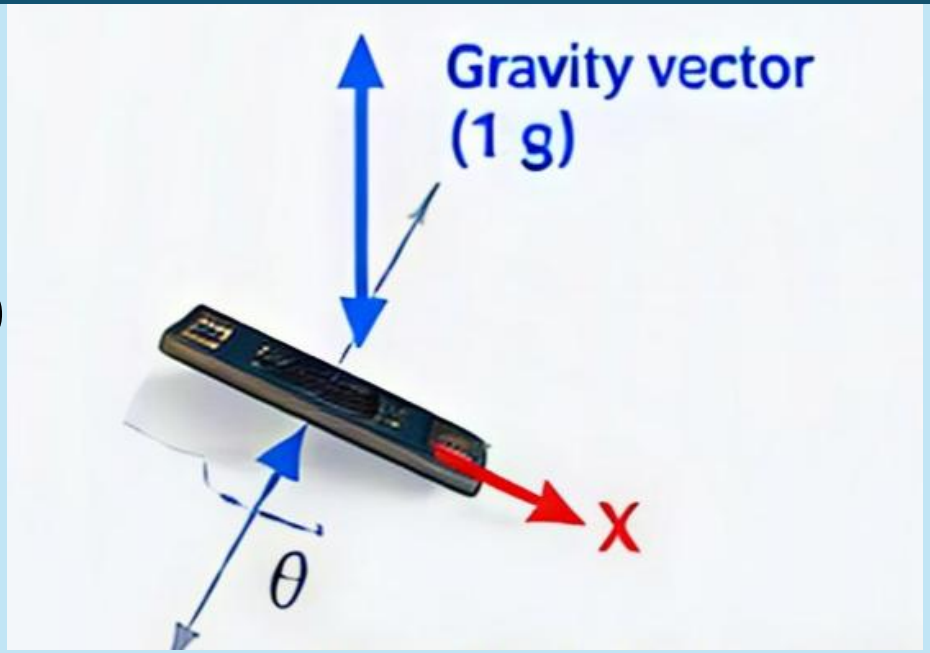
A_x: Linear acceleration along the X-axis (forward/backward tilt).

A_y: Linear acceleration along the Y-axis (side-to-side tilt).

A_z: Linear acceleration along the Z-axis (pointing straight up/down).

The Gravity and Measurement

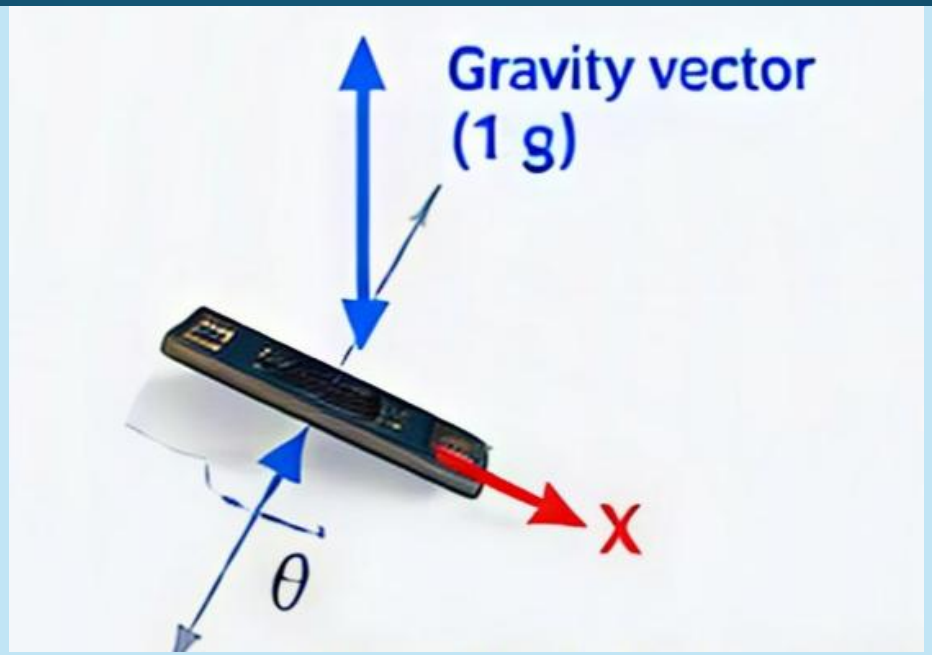
In 2-D assuming the $A_y = 0$



$$\theta_x = \arctan\left(\frac{A_x}{A_z}\right) \times \frac{180}{\pi}$$

The Gravity and Measurement

In 3D assuming the $A_y \neq 0$



$$\theta_x = \arctan\left(\frac{A_x}{\sqrt{A_y^2 + A_z^2}}\right) \times \frac{180}{\pi}$$

Let Try to Find the tilt in X-Y-Z

In MPU-6050 sensor is at rest initially ($\text{Roll}_0 = 0°, \text{Pitch}_0 = 0°$). At time $t = 0.05$ s, the sensor records the following values:

Accelerometer Readings (g):

- $A_x = 0.500\ g$
- $A_y = -0.300\ g$
- $A_z = 0.812\ g$

$\text{Roll}_0 = ?\ , \text{Pitch}_0 = ?)$

Let Try to Find the tilt in X-Y-Z

In MPU-6050 sensor is at rest initially ($\text{Roll}_0 = 0°, \text{Pitch}_0 = 0°, \text{Yaw}_0 = 0°$). At time $t = 0.05$ s, the sensor records the following values:

Accelerometer Readings (g):

- $A_x = 0.500\ g$
- $A_y = -0.300\ g$
- $A_z = 0.812\ g$

Pitch_x

$$\theta_a = \arctan\left(\frac{A_x}{\sqrt{A_y^2 + A_z^2}}\right) \times \frac{180}{\pi} = 0.98°$$

Let Try to Find the tilt in X-Y-Z

In MPU-6050 sensor is at rest initially ($\text{Roll}_0 = 0°, \text{Pitch}_0 = 0°, \text{Yaw}_0 = 0°$). At time $t = 0.05$ s, the sensor records the following values:

Accelerometer Readings (g):

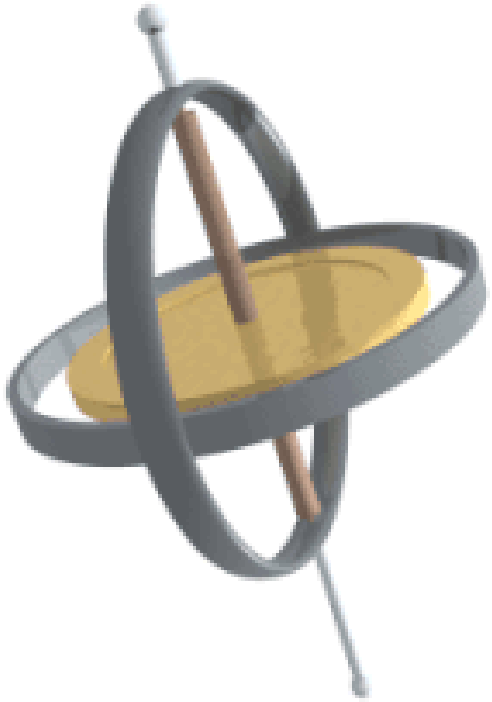
- $A_x = 0.500\ g$
- $A_y = -0.300\ g$
- $A_z = 0.812\ g$

ROLL_y $(\phi_y) = \arctan\left(\frac{-A_y}{\sqrt{A_x^2 + A_z^2}}\right) \times \frac{180}{\pi} = 1.18°$

Sensitivity Table

The raw accelerometer output is a signed 16-bit integer ranging from **-32,768 to +32,767**.

Full-Scale Range	Sensitivity Scale Factor	Resolution per LSB (g)	Resolution per LSB (m/s2)	Formula to Convert Raw to g
$\pm 2g$(Default)	16,384 LSB/g	0.0000610g/LSB	0.000598 m/s^2 /LSB	$A_g = \frac{\text{Raw Output}}{16384}$
$\pm 4g$	8,192 LSB/g	0.0001221g/LSB	0.001197 m/s^2 /LSB	$A_g = \frac{\text{Raw Output}}{8192}$
$\pm 8g$	4,096 LSB/g	0.0002441g/LSB	0.002395 m/s^2 /LSB	$A_g = \frac{\text{Raw Output}}{4096}$
$\pm 16g$	2,048 LSB/g	0.0004883g/LSB	0.004789 m/s^2 /LSB	$A_g = \frac{\text{Raw Output}}{2048}$



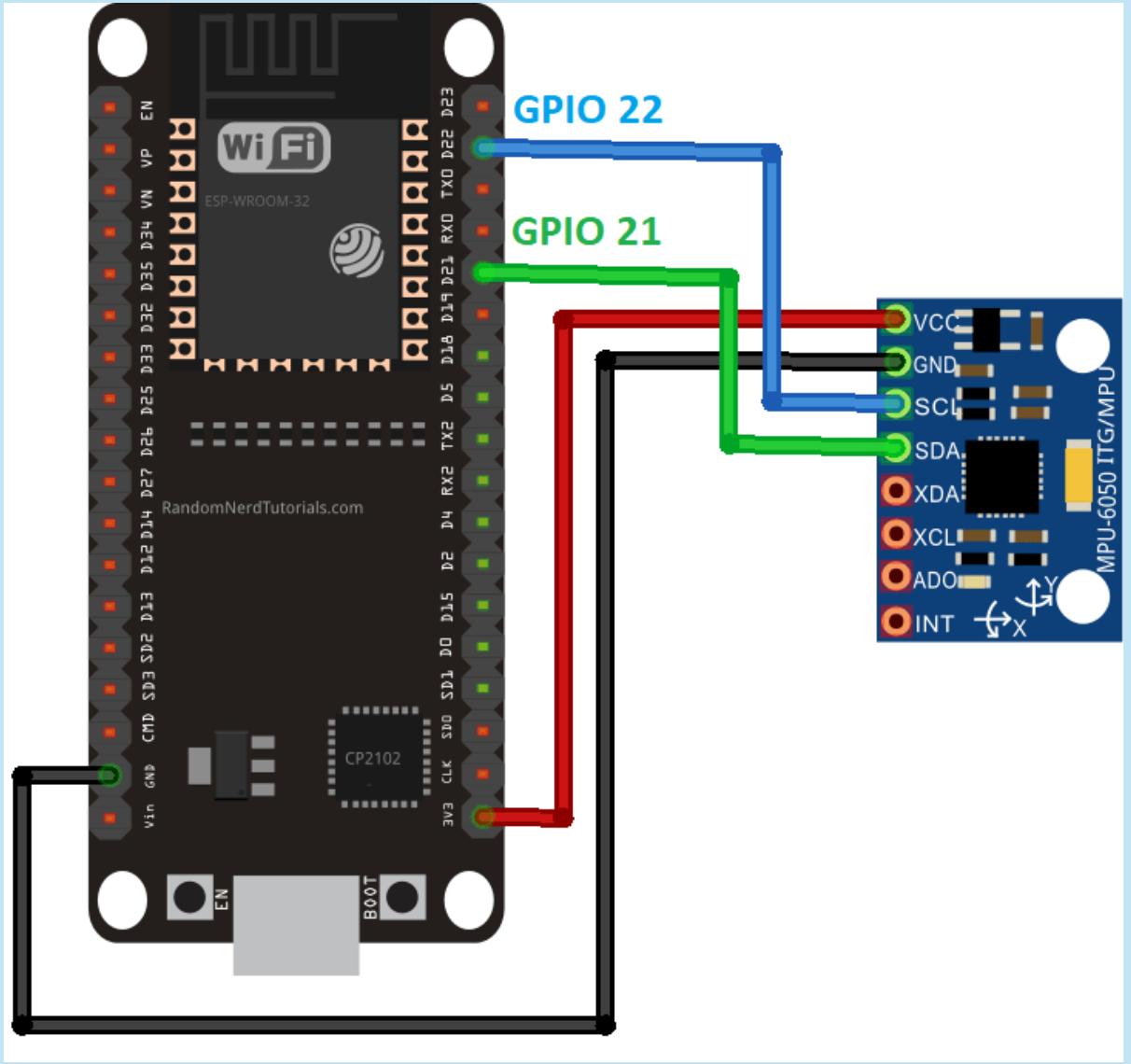
This Photo by Unknown Author is licensed under CC BY-SA

Gyroscope Table

The gyroscope outputs a signed 16-bit integer ($-32{,}768$ to $+32{,}767$, spanning 65,536 LSBs total).

Full-Scale Range	Sensitivity Scale Factor	Resolution per LSB (°/s)	Formula to Convert Raw to °/s
± 250 °/s(Default)	131.0 LSB/(°/s)	0.007633 °/s/LSB	$\omega = \frac{\text{Raw Output}}{131.0}$
± 500 °/s	65.5 LSB/(°/s)	0.015267 °/s/LSB	$\omega = \frac{\text{Raw Output}}{65.5}$
± 1000 °/s	32.8 LSB/(°/s)	0.030488 °/s/LSB	$\omega = \frac{\text{Raw Output}}{32.8}$
± 2000 °/s	16.4 LSB/(°/s)	0.060976 °/s/LSB	$\omega = \frac{\text{Raw Output}}{16.4}$

The Connection



Let’s Setup

I2C Setup

Step 1:

```
#include <Wire.h>
#include<math.h>

// MPU-6050 Standard I2C Address (0x68)
const uint8_t MPU6050_ADDR = 0x68;

// Register Addresses
const uint8_t REG_CONFIG      = 0x1A; // Low Pass Filter
const uint8_t REG_GYRO_CONFIG  = 0x1B; // Gyro Range
const uint8_t REG_ACCEL_CONFIG = 0x1C; // Accel Range
const uint8_t REG_PWR_MGMT_1   = 0x6B; // Power Management
const uint8_t REG_ACCEL_XOUT_H = 0x3B; // Start of Burst Read Data
```

I2C Setup

Step 2:

```
void setup() {
  Serial.begin(115200);

  // 1. Initialize I2C Pins (SDA = GPIO 21, SCL = GPIO 22 on ESP32)
  Wire.begin(21, 22);

  // 2. Set I2C Clock Speed Mode
  // Options: 100000 (Standard Mode 100 kHz) | 400000 (Fast Mode 400 kHz)
  Wire.setClock(100000);
```

I2C Setup

Step 3:

```
// 3. Wake up MPU-6050 (Set PWR_MGMT_1 to 0x00 to exit sleep mode)
 Wire.beginTransmission(MPU6050_ADDR);
 Wire.write(REG_PWR_MGMT_1);
 Wire.write(0x00);
 Wire.endTransmission();
```

I2C Setup

Step 4 (optional):

```
// 4. Configure Digital Low Pass Filter (DLPF) to 42Hz (~4.9ms delay)
  Wire.beginTransmission(MPU6050_ADDR);
  Wire.write(REG_CONFIG);
  Wire.write(0x03);
  Wire.endTransmission();
```

I2C Setup

Step 5:

```
// 5. Set Accelerometer Range to +/- 2g (0x00) -> 16384 LSB/g
Wire.beginTransmission(MPU6050_ADDR);
Wire.write(REG_ACCEL_CONFIG);
Wire.write(0x00);
Wire.endTransmission();
```

I2C Setup

Step 5 (optional):

```
// 6. Set Gyroscope Range to +/- 250 deg/s (0x00) -> 131 LSB/(deg/s)
  Wire.beginTransmission(MPU6050_ADDR);
  Wire.write(REG_GYRO_CONFIG);
  Wire.write(0x00);
  Wire.endTransmission();

  Serial.println("MPU-6050 Configured Successfully!");
}
```

Read the Data

Let’s Read Data

Step 1 :

```
void loop() {
    // 1. Point to the starting register (ACCEL_XOUT_H)
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(REG_ACCEL_XOUT_H);
    Wire.endTransmission(false); // Send restart to keep connection alive
    // 2. Request 14 consecutive bytes (Accel 6B + Temp 2B + Gyro 6B)
    Wire.requestFrom(MPU6050_ADDR, (uint8_t)14, (uint8_t)true);
```

Let’s Read Data

Step 2 :

```
if (Wire.available() >= 14) {
   // Combine High and Low Bytes (16-bit signed integers)
      int16_t rawAx = (Wire.read() << 8) | Wire.read();
      int16_t rawAy = (Wire.read() << 8) | Wire.read();
      int16_t rawAz = (Wire.read() << 8) | Wire.read();
```

Let’s Read Data

Step 3 :

```
int16_t rawTemp = (Wire.read() << 8) | Wire.read();
int16_t rawGx = (Wire.read() << 8) | Wire.read();
int16_t rawGy = (Wire.read() << 8) | Wire.read();
int16_t rawGz = (Wire.read() << 8) | Wire.read();
```

Let’s Read Data

Step 3 :

```
// 3. Convert Raw Values using Sensitivity Factors (+/-2g and +/-250dps)    float axg = rawAx / 16384.0;
float ayg = rawAy / 16384.0;
float azg = rawAz / 16384.0;

float gyroX = rawGx / 131.0;
float gyroY = rawGy / 131.0;
float gyroZ = rawGz / 131.0;

float tempC = (rawTemp + 521) / 340.0 + 35.0;
```

Let's Read Data

Step 4 (use math.h):

```
// Calculate Roll and Pitch angles in degrees from Accelerometer
      float pitch = atan2(Ax, sqrt(Ay * Ay + Az * Az)) * (180.0 / M_PI);
      float roll  = atan2(-Ay, sqrt(Ax * Ax + Az * Az)) * (180.0 / M_PI);

      // Print calculated Roll and Pitch to Serial Monitor    Serial.print("Roll: ");
      Serial.print(roll, 2);
      Serial.print(“,");
      Serial.print("Pitch: ");
      Serial.print(pitch, 2);
      Serial.println(" deg");
      // Please replace the text by ‘,’ if you want to visualise on the scope,
      }
delay(20); // 50 Hz loop
}
```

Let's Read Data

Step 4 (optional):

```
// 2. Complementary Filter Formula
// Pitch uses Gyro Y rate, Roll uses Gyro X rate
alpha = 0.96
pitch = alpha * (pitch + gyroY * dt) + (1-alpha) * accelPitch;
roll  = alpha * (roll  + gyroX * dt) + (1-alpha) * accelRoll;
```

$$f_c = \frac{1 - \alpha}{2\pi \cdot \alpha \cdot dt}$$

Let's Read Data

Step 4 (optional):

```
// 2. Complementary Filter Formula
// Pitch uses Gyro Y rate, Roll uses Gyro X rate
alpha = 0.96
pitch = alpha * (pitch + gyroY * dt) + (1-alpha) * accelPitch;
roll  = alpha * (roll  + gyroX * dt) + (1-alpha) * accelRoll;
```

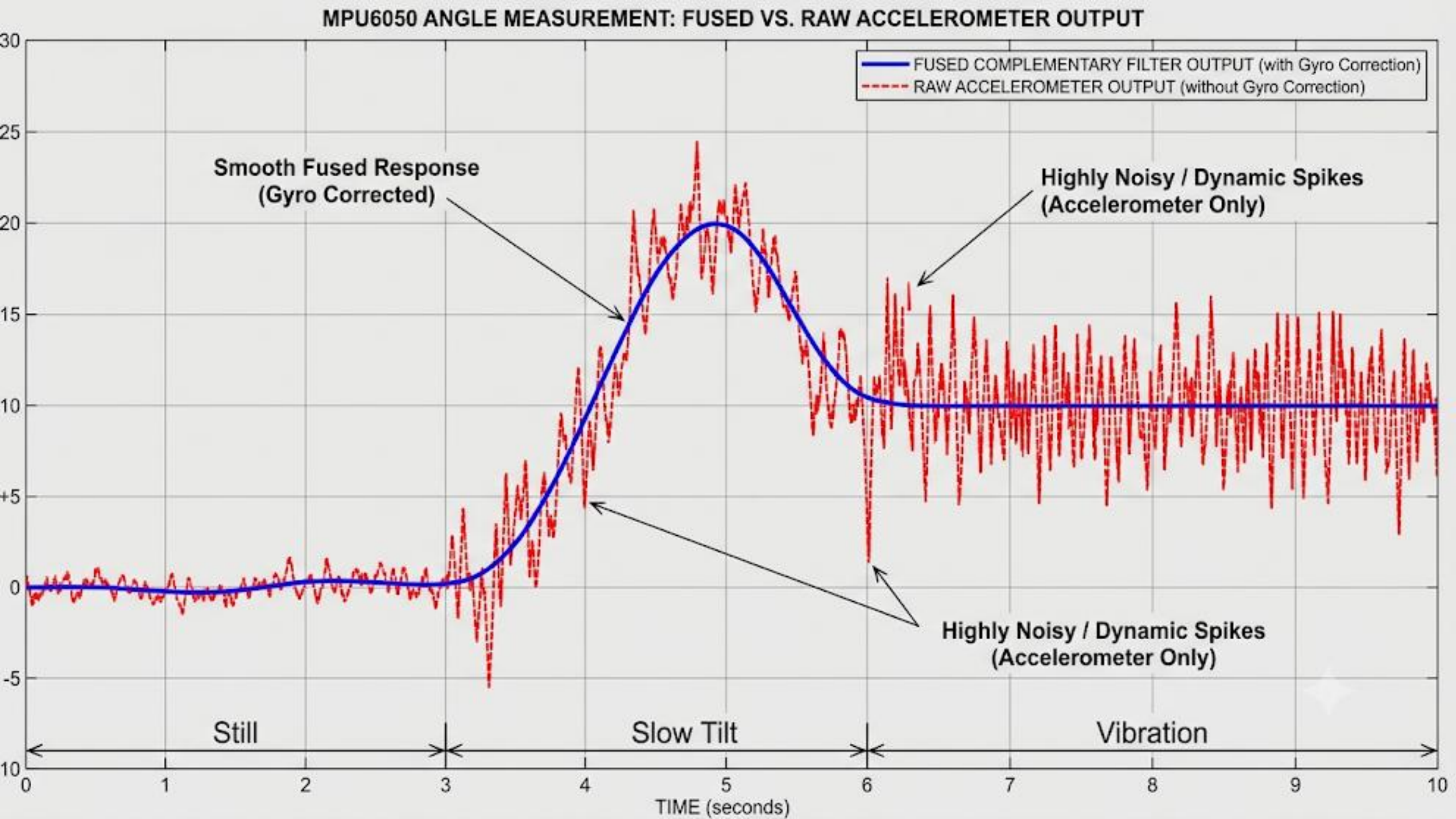
$$f_c = \frac{1 - 0.98}{2\pi \cdot 0.98 \cdot 0.01} = 0.325°/sec$$

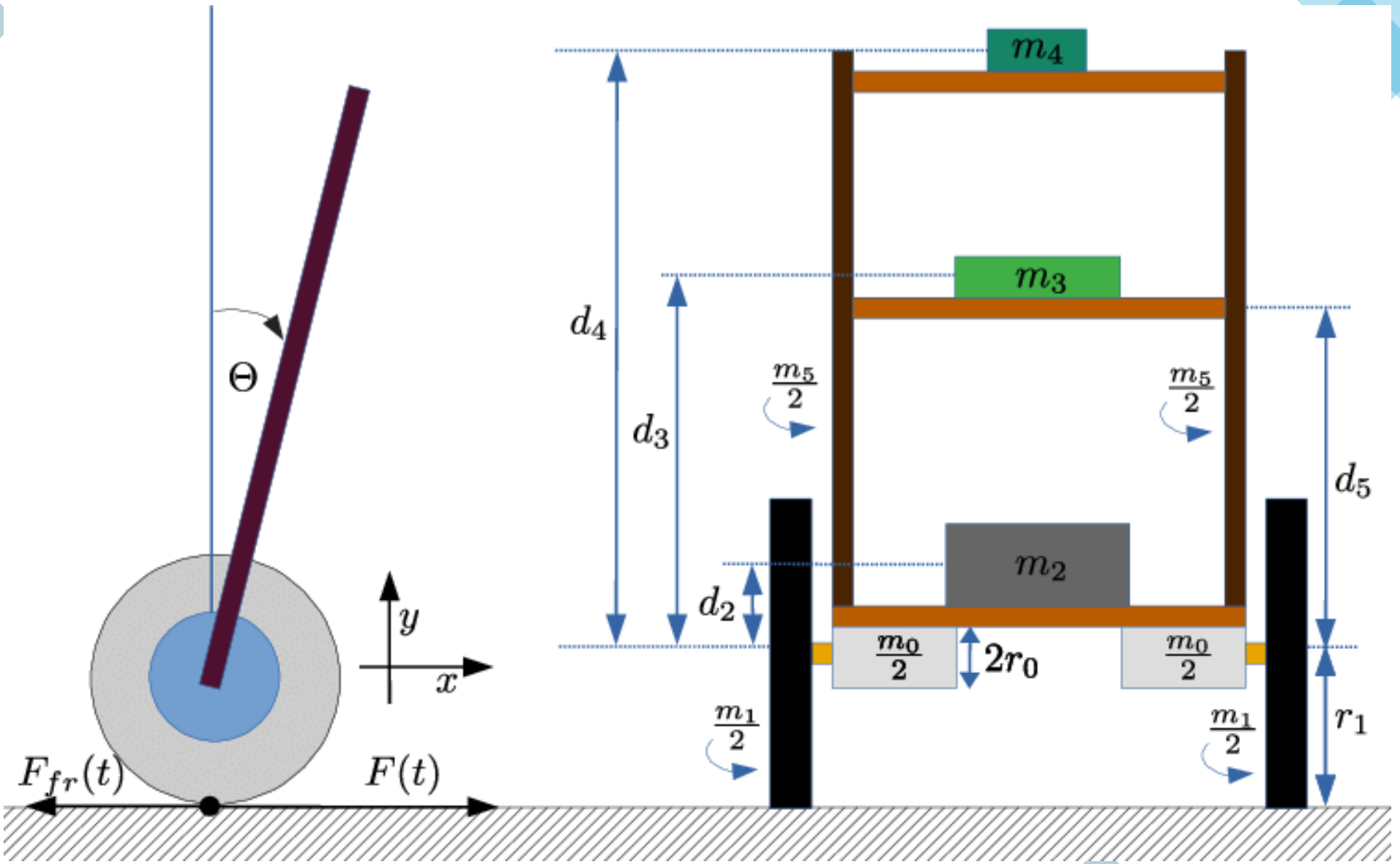
Let's Read Data

Step 4 (optional):

```
// 2. Complementary Filter Formula
// Pitch uses Gyro Y rate, Roll uses Gyro X rate
alpha = 0.96
pitch = alpha * (pitch + gyroY * dt) + (1-alpha) * accelPitch;
roll  = alpha * (roll  + gyroX * dt) + (1-alpha) * accelRoll;
```

*Any vibrations, tremors, or bumps happening faster than **0.325 times per second** are fully blocked out from the **accelerometer**. The **gyroscope** handles 100% of tracking for any motion faster than this threshold.*


MPU6050 ANGLE MEASUREMENT: FUSED VS. RAW ACCELEROMETER OUTPUT
FUSED COMPLEMENTARY FILTER OUTPUT (with Gyro Correction)
RAW ACCELEROMETER OUTPUT (without Gyro Correction)
Smooth Fused Response
(Gyro Corrected)
Highly Noisy / Dynamic Spikes
(Accelerometer Only)
Highly Noisy / Dynamic Spikes
(Accelerometer Only)
Still
Slow Tilt
Vibration
30
25
20
15
10
+5
0
-5
10
0
1
2
3
4
5
6
7
8
9
10
TIME (seconds)


m_4
m_3
m_2
d_4
d_3
d_2
d_5
$\frac{m_5}{2}$
$\frac{m_5}{2}$
$\frac{m_0}{2}$
$2r_0$
$\frac{m_0}{2}$
$\frac{m_1}{2}$
$\frac{m_1}{2}$
r_1
Θ
y
x
$F_{fr}(t)$
$F(t)$